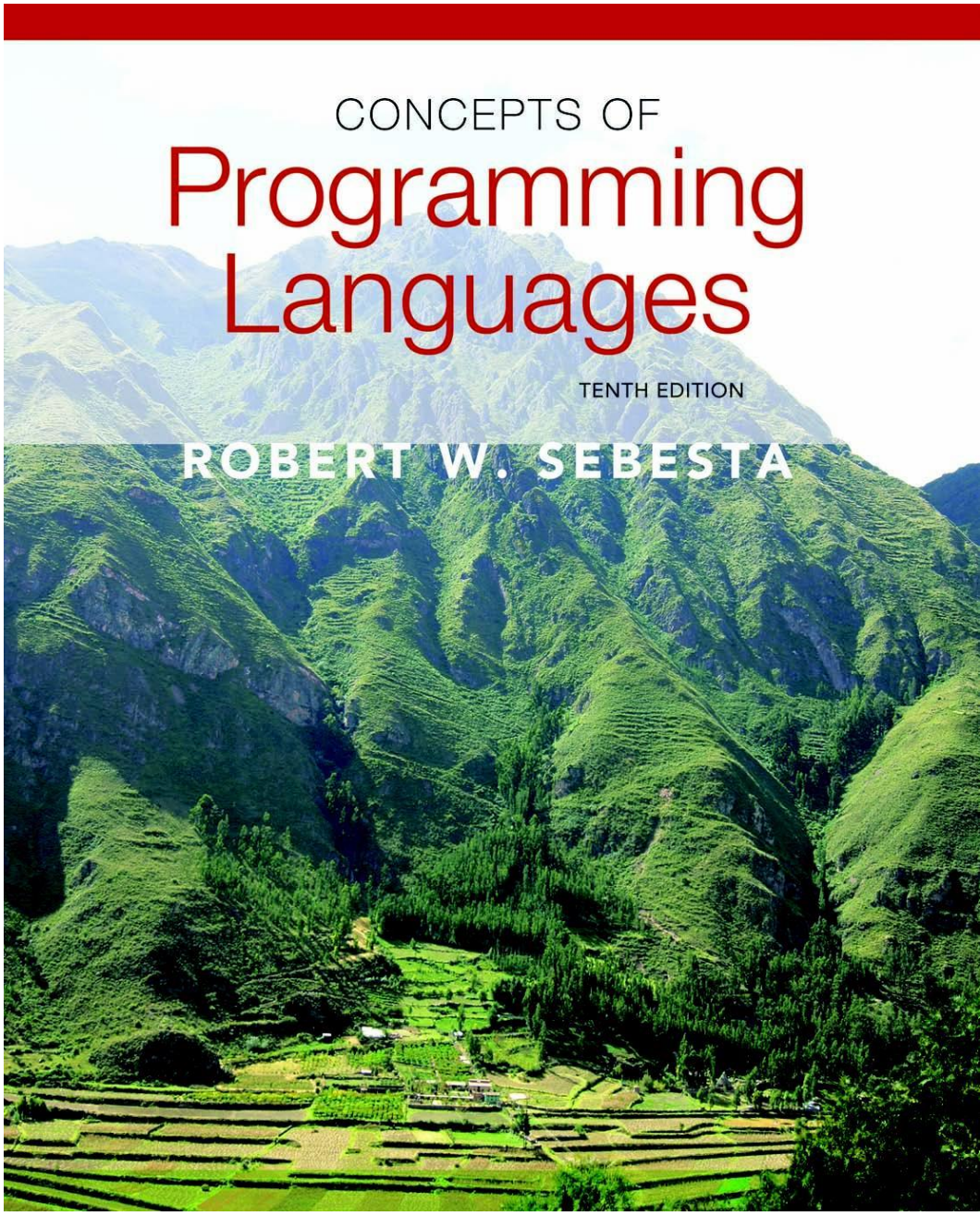
Chapter 6

Data Types



Introduction

- A *data type* defines a collection of data values and a set of predefined operations on those values
- A *descriptor* is the collection of the attributes of a variable
- An *object* represents an instance of a user-defined (abstract data) type
- One design issue for all data types:
  - What operations are defined and how are they specified?

Copyright © 2012 Addison-Wesley. All rights reserved.

Primitive Data Types

- Almost all programming languages provide a set of *primitive data types*
- Primitive data types: Those not defined in terms of other data types

C primitive types:

- `int, char, long, short, float, double, pointers`

Haskell primitive types:

- `Bool, Int, Float, Char, String, function values`

Python primitive types:

- `bool, int, float, complex, str, functions`

Copyright © 2012 Addison-Wesley. All rights reserved.

Primitive Data Types: Integer

- Almost always an exact reflection of the hardware so the mapping is trivial

- Java's signed integers (cardinality):

  `byte` (2^8), **`short`** (2^{16}), **`int`** (2^{32}), **`long`** (2^{64})

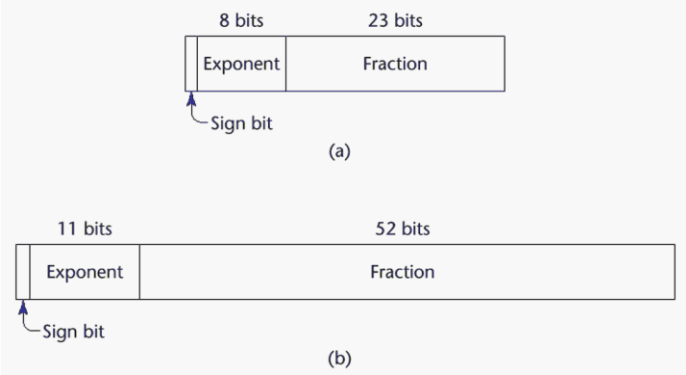
  - Cardinality refers to the number of basic members in a set.

- Some languages include unsigned integer types, e.g. C#, C++

Copyright © 2012 Addison-Wesley. All rights reserved.

Primitive Data Types: Floating Point

- Model real numbers, but only as approximations
- Languages for scientific use support at least two floating-point types (e.g., **`float`** and **`double`**) sometimes more
- Usually exactly like the hardware, but not always
- IEEE Floating-Point Standard 754
  - (a) single precision
  - (b) double precision



Copyright © 2012 Addison-Wesley. All rights reserved.

Primitive Data Types: Complex

- Some languages support a complex type, e.g., C99, Fortran, and Python
- Each value consists of two floats, the real part and the imaginary part
- Literal form (in Python):

  `(7 + 3j)`, where `7` is the real part and `3` is the imaginary part

Copyright © 2012 Addison-Wesley. All rights reserved.

Primitive Data Types: Decimal

- For business applications (money)
  - Essential to COBOL
  - C# offers a decimal data type
- Store a fixed number of decimal digits, in coded form (BCD)
- *Advantage*: accuracy
- *Disadvantages*: limited range, wastes memory

Copyright © 2012 Addison-Wesley. All rights reserved.

Primitive Data Types: Boolean

- Simplest of all
- Range of values: two elements, one for "true" and one for "false"
- Could be implemented as bits, but often as bytes
  - Advantage: readability

Copyright © 2012 Addison-Wesley. All rights reserved.

Primitive Data Types: Character

- Stored as numeric codings
- Most commonly used coding: ASCII
- An alternative,16-bit coding: Unicode (UCS-2)
  - Includes characters from most natural languages
  - Originally used in Java
  - C# and JavaScript also support Unicode
- 32-bit Unicode (UCS-4)
  - Supported by Fortran, starting with 2003

Copyright © 2012 Addison-Wesley. All rights reserved.

Character String Types

- Values are sequences of characters

- Design issues:
  - Is it a primitive type or just a special kind of array?
  - Should the length of strings be static or dynamic?

Copyright © 2012 Addison-Wesley. All rights reserved.

Character String Types Operations

- Typical operations:
  - Assignment and copying
  - Comparison (=, >, etc.)
  - Catenation
  - Substring reference
  - Pattern matching

Copyright © 2012 Addison-Wesley. All rights reserved.

Character String Type in Certain Languages

- C and C++
  - Not primitive
  - Use **`char`** arrays and a library of functions that provide operations
- SNOBOL4 (a string manipulation language)
  - Primitive
  - Many operations, including complex pattern matching
- Fortran and Python
  - Primitive type with assignment and several operations
- Java
  - Primitive via the `String` class
- Perl, JavaScript, Ruby, and PHP
  - Provide built-in pattern matching, using regular expressions

Copyright © 2012 Addison-Wesley. All rights reserved.

Character String Length Options

- Static: Python, Java's `String` class
- *Limited Dynamic Length*: C and C++
  - In these languages, a special character is used to indicate the end of a string's characters, rather than maintaining the length
- *Dynamic* (no maximum): SNOBOL4, Perl, JavaScript
- Ada supports all three string length options

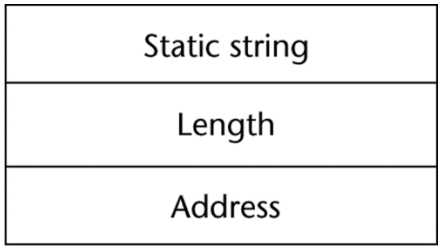
Copyright © 2012 Addison-Wesley. All rights reserved.

Character String Implementation

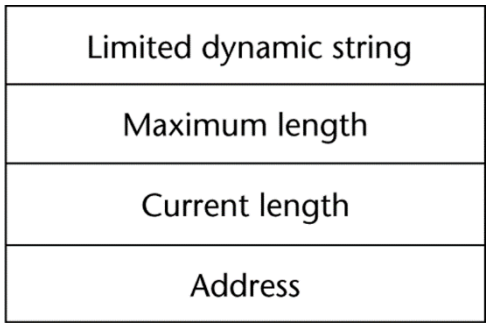
- Static length: compile-time descriptor
- Limited dynamic length: may need a run-time descriptor for length (but not in C and C++ (\0))
- Dynamic length: need run-time descriptor; allocation/deallocation is the biggest implementation problem

Copyright © 2012 Addison-Wesley. All rights reserved.

Compile- and Run-Time Descriptors



Compile-time descriptor for static strings



Run-time descriptor for limited dynamic strings

Copyright © 2012 Addison-Wesley. All rights reserved.

User-Defined Ordinal Types

- An ordinal type is one in which the range of possible values can be easily associated with the set of positive integers

- Two user-defined ordinal types:
  - Enumeration
  - Subrange

Copyright © 2012 Addison-Wesley. All rights reserved.

Enumeration Types

- All possible values, which are named constants, are provided in the definition
- C# example

```
enum days {mon, tue, wed, thu, fri, sat, sun};
```

- Design issues
  - Is an enumeration constant allowed to appear in more than one type definition, and if so, how is the type of an occurrence of that constant checked?
  - Are enumeration values coerced to integer?
  - Any other type coerced to an enumeration type?

Copyright © 2012 Addison-Wesley. All rights reserved.

Evaluation of Enumerated Type

- Aid to readability, e.g., no need to code a color as a number

- Aid to reliability, e.g., compiler can check:
  - operations (don't allow colors to be added)
  - No enumeration variable can be assigned a value outside its defined range
  - Ada, C#, and Java 5.0 provide better support for enumeration than C++ because enumeration type variables in these languages are not coerced into integer types

Copyright © 2012 Addison-Wesley. All rights reserved.

Subrange Types



- An ordered contiguous subsequence of an ordinal type
  - Example: 12..18 is a subrange of integer type
- Included in Pascal and Ada
- Ada's design

```
type Days is (mon, tue, wed, thu, fri, sat, sun);
   subtype Weekdays is Days range mon..fri;
   subtype Index is Integer range 1..100;

   Day1: Days;
   Day2: Weekdays;
   Day2 := Day1; (Day1 should not be sat or sun)
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Subrange Evaluation

- Aid to readability
  - Make it clear to the readers that variables of subrange can store only certain range of values
- Reliability
  - Assigning a value to a subrange variable that is outside the specified range is detected as an error

Copyright © 2012 Addison-Wesley. All rights reserved.

Implementation of User-Defined Ordinal Types

- Enumeration types are implemented as integers
- Subrange types are implemented like their parent types with code inserted (by the compiler) to restrict assignments to subrange variables

Copyright © 2012 Addison-Wesley. All rights reserved.

Array Types

- An array is a homogeneous aggregate of data elements in which an individual element is identified by its position in the aggregate, relative to the first element.

Copyright © 2012 Addison-Wesley. All rights reserved.

Array Design Issues

- What types are legal for subscripts?
- Are subscripting expressions in element references range checked?
- When are subscript ranges bound?
- When does allocation take place?
- What is the maximum number of subscripts?
- Can array objects be initialized?
- Are any kind of slices supported?

Copyright © 2012 Addison-Wesley. All rights reserved.

Array Indexing

- *Indexing* (or subscripting) is a mapping from indices to elements

  array_name [index_value] → an element

- Index Syntax
  - Fortran and Ada use parentheses
  - Most other languages use brackets

Copyright © 2012 Addison-Wesley. All rights reserved.

Arrays Index (Subscript) Types

- FORTRAN, C: integer only
- Java: integer types only
- Pascal: integer, enumeration, Boolean, char

```
type
     Day = Mon, Tue, Wed, Thu, Fri, Sat, Sun;
var
     x : array Day of real;
     …
     x[Tue] := 2.4;
```

- Index range checking
  - C, C++, Perl, and Fortran do not specify range checking
  - Java, ML, C# specify range checking
  - In Ada, the default is to require range checking, but it can be turned off

Copyright © 2012 Addison-Wesley. All rights reserved.

Subscript Binding and Array Categories

- ***Static:*** subscript ranges are statically bound and storage allocation is static (before run-time)

- ***Dynamic***: subscript ranges and the allocation are done at run-time
  - Fixed stack-dynamic (C, C++ arrays)
  - Fixed heap-dynamic (C, C++ dynamic arrays, Java non-generic arrays)
  - Heap-dynamic (Java ArrayList, C# List)

Copyright © 2012 Addison-Wesley. All rights reserved.

Subscript Binding and Array Categories

- C and C++ support static arrays
- Java, JavaScript, Python, Ruby support dynamic arrays

Java non-generic arrays:

```
int[] numbers = {1, 2, 3};
String[] names = {"Alice", "Bob"};
Object[] things = {1, "text", 3.14};
```

Java Generic arrays:

```
List<String> names = new ArrayList<>();
names.add("Alice");
```

Python arrays

```
numbers = []
for i in range(5):
    numbers.append(i)
print(numbers)  # [0, 1, 2, 3, 4]
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Array Initialization

- Some languages allow initialization at the time of storage allocation
  - C, C++, Java, C# example

```
int list [] = {4, 5, 7, 83}
```

  - Character strings in C and C++

```
char name [] = "freddie";
```

  - Arrays of strings in C and C++

```
char *names [] = {"Bob", "Jake", "Joe"];
```

  - Java initialization of String objects

```
String[] names = {"Bob", "Jake", "Joe"};
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Array Initialization

- C-based languages

```
int list [] = {1, 3, 5, 7}
char *names [] = {"Mike", "Fred", "Mary Lou"};
```

- Python
  - List comprehensions

```
list = [x ** 2 for x in range(12) if x % 3 == 0]
```

puts [0, 9, 36, 81] in list

Copyright © 2012 Addison-Wesley. All rights reserved.

Heterogeneous Arrays

- A *heterogeneous array* is one in which the elements need not be of the same type
- Supported by Perl, Python, JavaScript, and Ruby

Copyright © 2012 Addison-Wesley. All rights reserved.

Array Operations

- C-based languages do not provide any array operations
- Ada allows array assignment and also catenation (&).
- Python allows array assignments, but only reference changes. Python also supports array catenation (+) and element membership (`in`) operations
- Ruby provides array catenation (`Array`)
- Fortran provides **elemental** operations because they are between pairs of array elements
  - For example, + operator between two arrays results in an array of the sums of the element pairs of the two arrays

Copyright © 2012 Addison-Wesley. All rights reserved.

Rectangular and Jagged Arrays

- A rectangular array is a multi-dimensioned array in which all of the rows have the same number of elements and all columns have the same number of elements
- A jagged matrix has rows with varying number of elements
- C, C++, and Java support jagged arrays

```
int *jagged[5];
jagged[0] = malloc(sizeof(int) * 10);
jagged[1] = malloc(sizeof(int) * 3);
```

- Fortran and Ada support rectangular arrays

Copyright © 2012 Addison-Wesley. All rights reserved.

Slices

- A slice is some substructure of an array
  - not a new data type
  - only a referencing mechanism
  - Ex: A is a matrix → first row of A is a slice
- Slices are only useful in languages that have array operations

Copyright © 2012 Addison-Wesley. All rights reserved.

Slice Examples

- Python

```
vector = [2, 4, 6, 8, 10, 12, 14, 16]
mat = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

`vector[3:6]` is a three-element array → `[8,10,12]`

`mat[0][0:2]` is the 1st and 2nd elem of the 1st row → `[1,2]`

- Ruby supports slices with the `slice` method
  - `list = [2, 4, 6, 8, 10]`
  - `list.slice(4)` returns `10`
  - `list.slice(2, 2)` returns `[6,8]`
  - `list.slice(1..3)` returns `[4,6,8]`

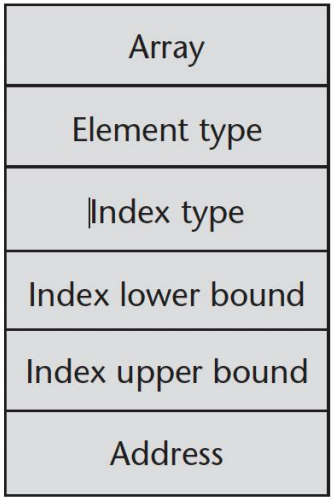
Copyright © 2012 Addison-Wesley. All rights reserved.

Implementation of Arrays

- Access function maps subscript expressions to an address in the array
- Access function for single-dimensioned arrays:

```
address(list[k]) = address(list[l_b])+ ((k-l_b)*el_size)
```

- Compile-time descriptor for a single-dimensioned array



Copyright © 2012 Addison-Wesley. All rights reserved.

Accessing Multi-dimensioned Arrays

- Two common ways:

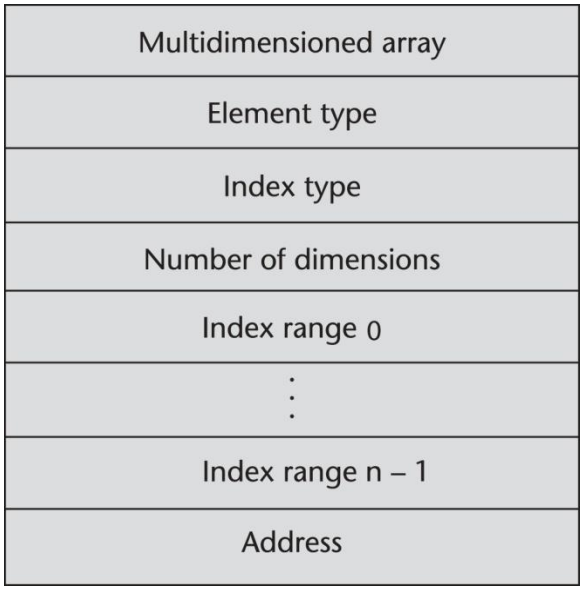
3	4	7
6	2	5
1	3	8

  - Row major order (by rows) – used in most languages
    - 3, 4, 7, 6, 2, 5, 1, 3, 8
  - Column major order (by columns) – used in Fortran
    - 3, 6, 1, 4, 2, 3, 7, 5, 8

Copyright © 2012 Addison-Wesley. All rights reserved.

Accessing Multi-dimensioned Arrays

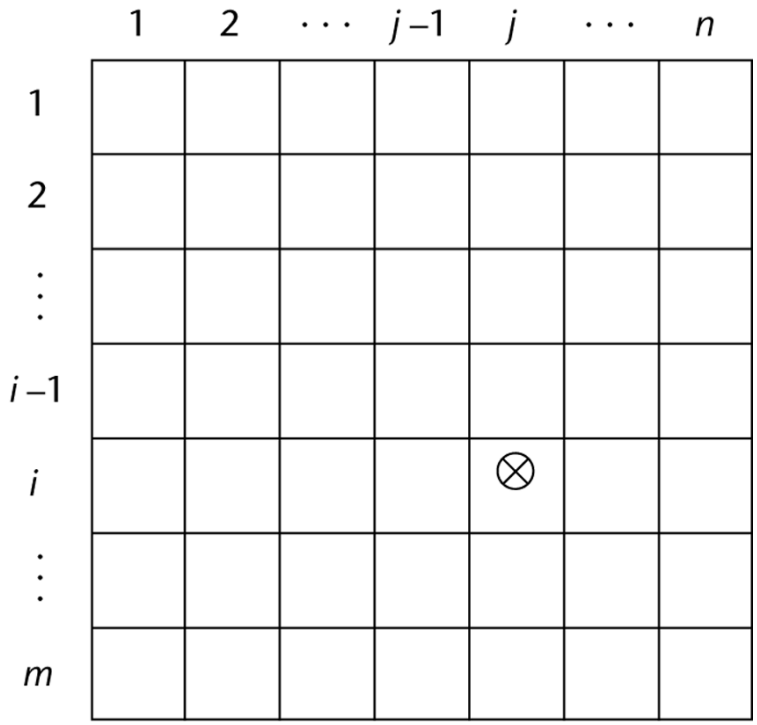
- Compile-time descriptor for a multi-dimensioned array



Copyright © 2012 Addison-Wesley. All rights reserved.

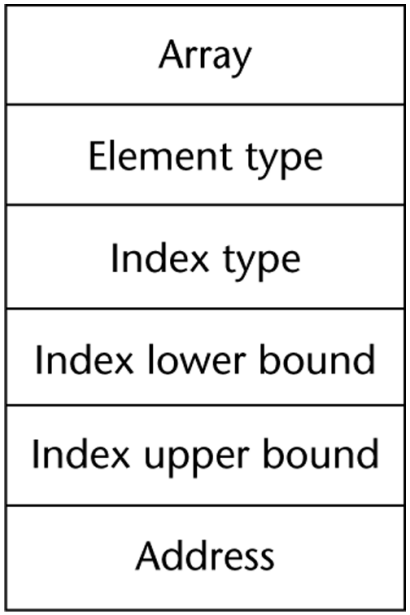
Locating an Element in a Multi-dimensioned Array

```
location (a[i,j]) = address of a[row_lb,col_lb]
                  + (((i-row_lb)*n) + (j-col_lb)) * e_size
```

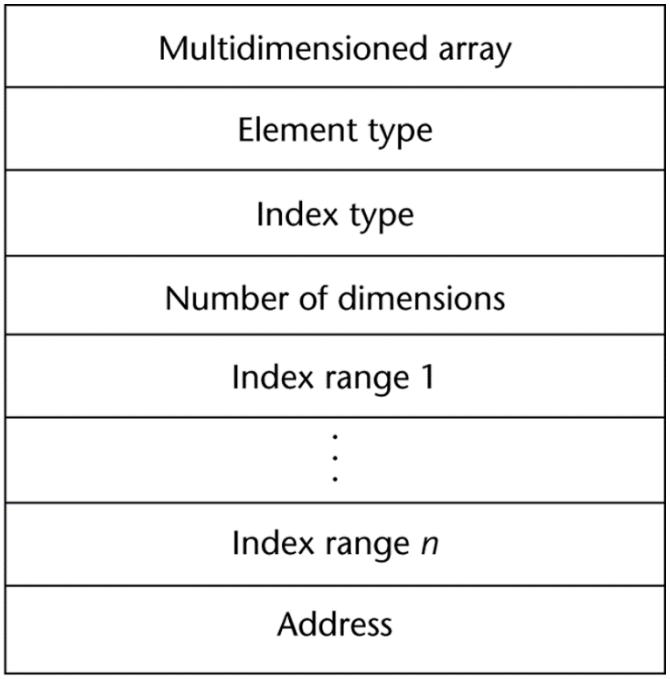


Copyright © 2012 Addison-Wesley. All rights reserved.

Compile-Time Descriptors



Single-dimensioned array



Multidimensional array

Copyright © 2012 Addison-Wesley. All rights reserved.

Associative Arrays

- An *associative array* is an unordered collection of data elements that are indexed by an equal number of values called *keys*
  - User-defined keys must be stored
- Built-in type in Perl, Python, Ruby, and Lua

Copyright © 2012 Addison-Wesley. All rights reserved.

Associative Arrays in Python (Dictionary)

```
# Define an associative array (dictionary)
person = {
    "name": "Alice",
    "age": 30,
    "city": "New York"
}

# Access values by key
print(person["name"])  # Output: Alice

# Add a new key-value pair
person["email"] = "alice@example.com"

# Modify a value
person["age"] = 31

# Check if a key exists
if "city" in person:
    print("City is:", person["city"])

# Loop through keys and values
for key, value in person.items():
    print(key, "=>", value)
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Associative Arrays in Java (Map)

```
Map<String, String> map = new HashMap<>();
map.put("fruit", "apple");
map.put("vegetable", "carrot");

System.out.println(map.get("fruit"));  // Output: apple
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Associative Arrays in Perl (hashes)

- Names begin with `%`; literals are delimited by parentheses

```
%hi_temps = ("Mon" => 77, "Tue" => 79, "Wed" =>65);
```

- Subscripting is done using braces and keys

```
$hi_temps{"Wed"} = 83;
```

- Elements can be removed with **`delete`**

```
delete $hi_temps{"Tue"};
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Record Types

- A *record* is a possibly heterogeneous aggregate of data elements in which the individual elements are identified by names

- Design issues:
  - What is the syntactic form of references to the field?
  - Are elliptical references allowed

Copyright © 2012 Addison-Wesley. All rights reserved.

Definition of Records in COBOL

- COBOL uses level numbers to show nested records; others use recursive definition

```
01 EMP-REC.
   02 EMP-NAME.
      05 FIRST PIC X(20).
      05 MID   PIC X(10).
      05 LAST  PIC X(20).
   02 HOURLY-RATE PIC 99V99.
```

Copyright © 2012 Addison-Wesley. All rights reserved.

References to Records

- Record field references
  1. COBOL

  field_name `OF` record_name_1 `OF` ... `OF` record_name_n

  e.g. MID OF EMP-NAME OF EMP-REC
  2. Others (dot notation)

  record_name_1.record_name_2. ... record_name_n.field_name

  e.g. Emp_Rec.Emp_Name.Mid

- Fully qualified references must include all record names

- Elliptical references allow leaving out record names as long as the reference is unambiguous, for example in COBOL `FIRST`, `FIRST OF EMP-NAME`, and `FIRST OF EMP-REC` are elliptical references to the employee's first name

Copyright © 2012 Addison-Wesley. All rights reserved.

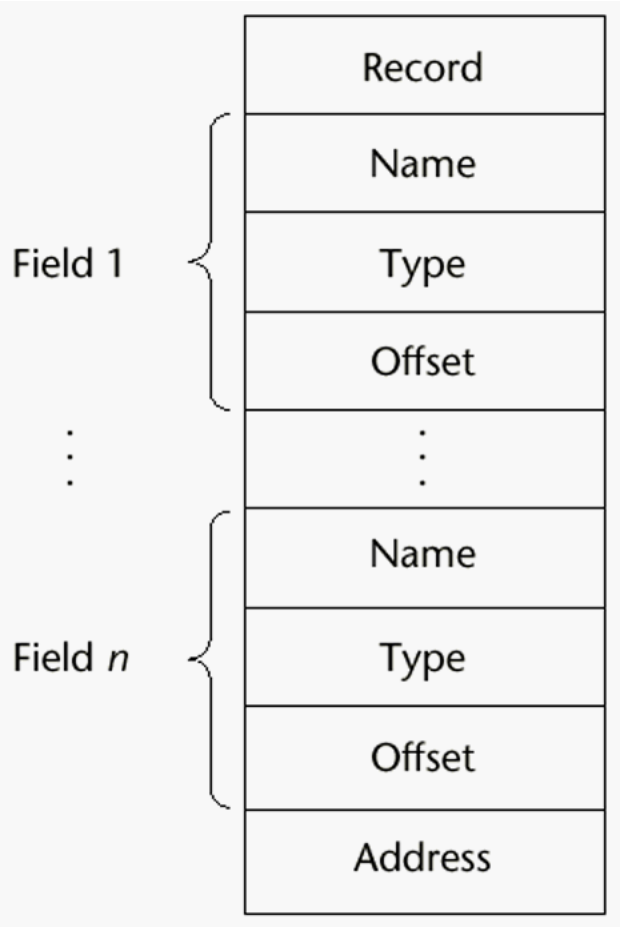
Evaluation and Comparison to Arrays

- Records are used when collection of data values is heterogeneous
- Access to array elements can be slower than access to record fields, because array subscripts are dynamic (resolved at runtime), whereas field names in records are typically static (known at compile time)

Copyright © 2012 Addison-Wesley. All rights reserved.

Implementation of Record Type

Offset address relative to the beginning of the records is associated with each field



Copyright © 2012 Addison-Wesley. All rights reserved.

Tuple Types

- A tuple is a data type that is similar to a record, except that the elements are not named
- Used in Python, ML, and F# to allow functions to return multiple values
  - Python
    - Closely related to its lists, but immutable
    - Create with a tuple literal

```
myTuple = (3, 5.8, 'apple')
```

    - Referenced with subscripts (begin at `0`)

`myTuple[0]` returns `3`

    - Catenation of tuples with `+`

Copyright © 2012 Addison-Wesley. All rights reserved.

Tuple Types (continued)

- Haskell

```
type Student=(Int,String)
```

  - Access as follows (for a tuple of size 2):

    `fst x` is the first element

    `snd x` is the second element

  - Assign value

```
let x=((123, "Ali")::Student)
```

Copyright © 2012 Addison-Wesley. All rights reserved.

List Types

- Lists in LISP and Scheme are delimited by parentheses and use no commas

  `(A B C D)` and `(A (B C) D)`

- Data and code have the same form

  As data, `(A B C)` is literally what it is

  As code, `(A B C)` is the function `A` applied to the parameters `B` and `C`

- The interpreter needs to know which one is a list, so if it is data, we quote it with an apostrophe

  `'(A B C)` is data

Copyright © 2012 Addison-Wesley. All rights reserved.

List Types (continued)

- List Operations in Scheme
  - `CAR` returns the first element of its list parameter

    `(CAR '(A B C))` returns `A`
  - `CDR` returns the remainder of its list parameter after the first element has been removed

    `(CDR '(A B C))` returns `(B C)`
  - `CONS` puts its first parameter into its second parameter, a list, to make a new list

    `(CONS 'A (B C))` returns `(A B C)`
  - `LIST` returns a new list of its parameters

    `(LIST 'A 'B '(C D))` returns `(A B (C D))`

Copyright © 2012 Addison-Wesley. All rights reserved.

List Types (continued)

- List Operations in ML
  - Lists are written in brackets and the elements are separated by commas
  - List elements must be of the same type
  - The Scheme `CONS` function is a binary operator in ML, `::`

    `3 :: [5, 7, 9]` evaluates to `[3, 5, 7, 9]`
  - The Scheme `CAR` and `CDR` functions are named `hd` and `tl`, respectively

Copyright © 2012 Addison-Wesley. All rights reserved.

List Types (continued)

- F# Lists
  - Like those of ML, except elements are separated by semicolons and `hd` and `tl` are methods of the `List` class
- Python Lists
  - The list data type also serves as Python's arrays
  - Elements can be of any type
  - Create a list with an assignment

```
myList = [3, 5.8, "grape"]
```

Copyright © 2012 Addison-Wesley. All rights reserved.

List Types (continued)

- Python Lists (continued)
  - List elements are referenced with subscripting, with indices beginning at zero

    `x = myList[1]` Sets `x` to `5.8`
  - List elements can be deleted with `del`

    `del myList[1]`
  - List Comprehensions – derived from set notation

    - *[expression **for** iterate_var **in** array **if** condition]*

    `[x * x for x in range(6) if x % 3 == 0]`

    `range(6)` creates `[0, 1, 2, 3, 4, 5]`

    Constructed list: `[0, 9]`

Copyright © 2012 Addison-Wesley. All rights reserved.

List Types (continued)

- Haskell's List Comprehensions
  - `[n * n | n <- [1..5]] defines [1,4,9,16,25]`
- F#'s List Comprehensions

```
let myArray = [|for i in 1 .. 5 -> (i * i) |];;
defines [1;4;9;16;25]
```

- Both C# and Java supports lists through their generic heap-dynamic collection classes, `List` and `ArrayList`, respectively

Copyright © 2012 Addison-Wesley. All rights reserved.

Unions Types

- A *union* is a type whose variables are allowed to store different type values at different times during execution
- Design issues
  - Should type checking be required?

Copyright © 2012 Addison-Wesley. All rights reserved.

Discriminated vs. Free Unions

- C and C++ provide union constructs in which there is no language support for type checking; the union in these languages is called *free union*
- Type checking of unions require that each union include a type indicator called a *discriminant*
  - Supported by ML, Haskell, and F#

Copyright © 2012 Addison-Wesley. All rights reserved.

Evaluation of Unions

- Free unions are unsafe
  - Do not allow type checking
- Java and C# do not support unions
  - Reflective of growing concerns for safety in programming languages

Copyright © 2012 Addison-Wesley. All rights reserved.

Pointer and Reference Types

- A *pointer* type variable has a range of values that consists of memory addresses and a special value, *nil*
- Provide the power of indirect addressing
- Provide a way to manage dynamic memory
- A pointer can be used to access a location in the area where storage is dynamically created (usually called a *heap*)

Copyright © 2012 Addison-Wesley. All rights reserved.

Design Issues of Pointers

- What are the scope of and lifetime of a pointer variable?
- What is the lifetime of a heap-dynamic variable?
- Are pointers restricted as to the type of value to which they can point?
- Are pointers used for dynamic storage management, indirect addressing, or both?
- Should the language support pointer types, reference types, or both?

Copyright © 2012 Addison-Wesley. All rights reserved.

Pointer Operations

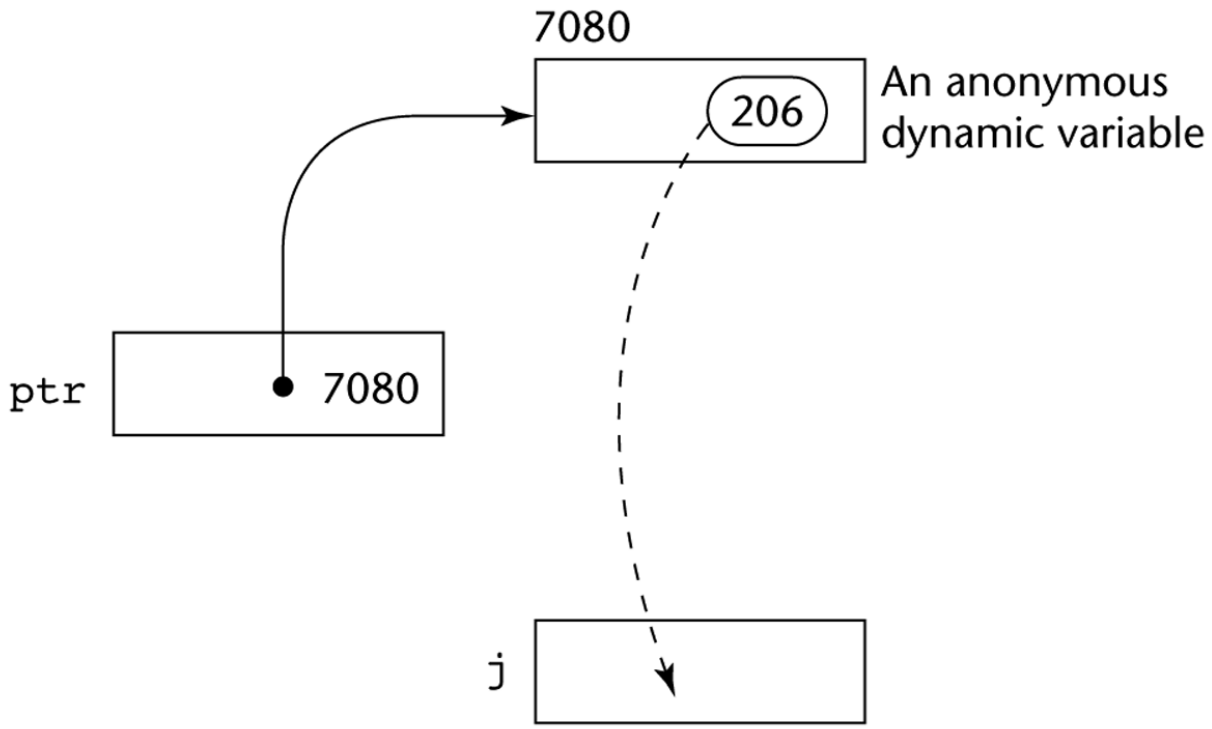
- Two fundamental operations: assignment and dereferencing
- Assignment is used to set a pointer variable's value to some useful address
- Dereferencing yields the value stored at the location represented by the pointer's value
  - Dereferencing can be explicit or implicit
  - C++ uses an explicit operation via *

    `j = *ptr`

    sets j to the value located at `ptr`

Copyright © 2012 Addison-Wesley. All rights reserved.

Pointer Assignment Illustrated



The assignment operation j = *ptr

Copyright © 2012 Addison-Wesley. All rights reserved.

Problems with Pointers

- Dangling pointers (dangerous)
  - A pointer points to a heap-dynamic variable that has been deallocated

```
int * arrayPtr1;
int * arrayPtr2 = new int[100];
arrayPtr1 = arrayPtr2;
delete [] arrayPtr2;
// Now, arrayPtr1 is dangling, because the heap storage
// to which it was pointing has been deallocated.
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Problems with Pointers

- Lost heap-dynamic variable
  - An allocated heap-dynamic variable that is no longer accessible to the user program (often called *garbage*)
    - Pointer `p1` is set to point to a newly created heap-dynamic variable
    - Pointer `p1` is later set to point to another newly created heap-dynamic variable
    - The process of losing heap-dynamic variables is called *memory leakage*

```
int * p1 = new int[100];
*p1 = new int[100];
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Pointers in C and C++

- Extremely flexible but must be used with care
- Pointers can point at any variable regardless of when or where it was allocated
- Used for dynamic storage management and addressing
- Pointer arithmetic is possible
- Explicit dereferencing and address-of operators
- Domain type need not be fixed (`void *`)

  `void *` can point to any type and can be type checked (cannot be de-referenced)

Copyright © 2012 Addison-Wesley. All rights reserved.

Pointer Arithmetic in C and C++

```
float stuff[100];
float *p;
p = stuff;
```

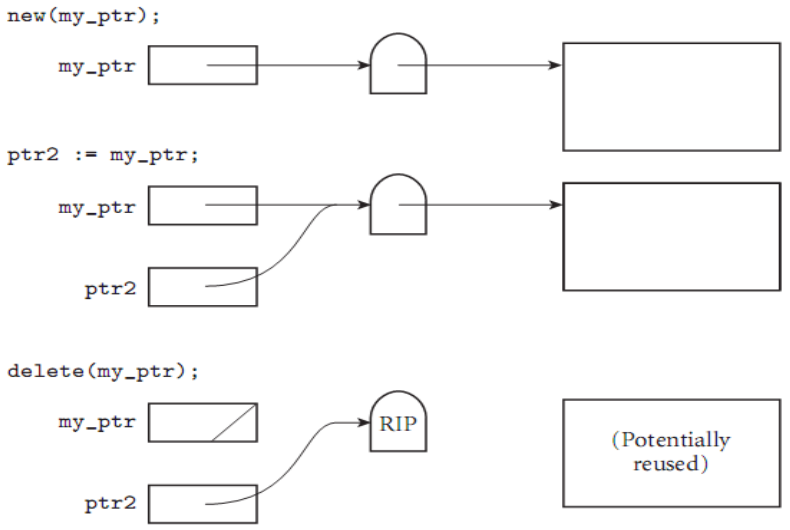
`*(p+5)` is equivalent to `stuff[5]` and `p[5]`

`*(p+i)` is equivalent to `stuff[i]` and `p[i]`

Copyright © 2012 Addison-Wesley. All rights reserved.

Dangling Pointer Problem

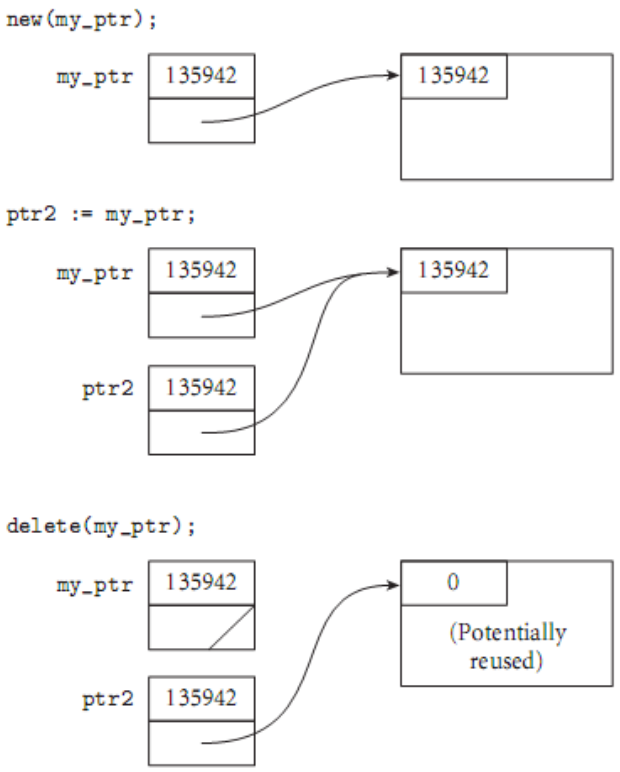
- *Tombstone*: extra heap cell that is a pointer to the heap-dynamic variable
  - The actual pointer variable points only at tombstones
  - When heap-dynamic variable de-allocated, tombstone remains but set to nil
  - Costly in time and space



Copyright © 2012 Addison-Wesley. All rights reserved.

Dangling Pointer Problem

- *Locks-and-keys*: Pointer values are represented as (key, address) pairs
  - Heap-dynamic variables are represented as variable plus cell for integer lock value
  - When heap-dynamic variable allocated, lock value is created and placed in lock cell and key cell of pointer



Copyright © 2012 Addison-Wesley. All rights reserved.

Reference Types

- C++ includes a special kind of pointer type called a *reference type* that is used primarily for formal parameters
  - Advantages of both pass-by-reference and pass-by-value
- Java extends C++'s reference variables and allows them to replace pointers entirely
  - References are references to objects, rather than being addresses
- C# includes both the references of Java and the pointers of C++

Copyright © 2012 Addison-Wesley. All rights reserved.

Heap Management

- A very complex run-time process
- Single-size cells vs. variable-size cells
- Two approaches to reclaim garbage
  - Reference counters (*eager approach*): reclamation is gradual
  - Mark-sweep (*lazy approach*): reclamation occurs when the list of variable space becomes empty

Copyright © 2012 Addison-Wesley. All rights reserved.

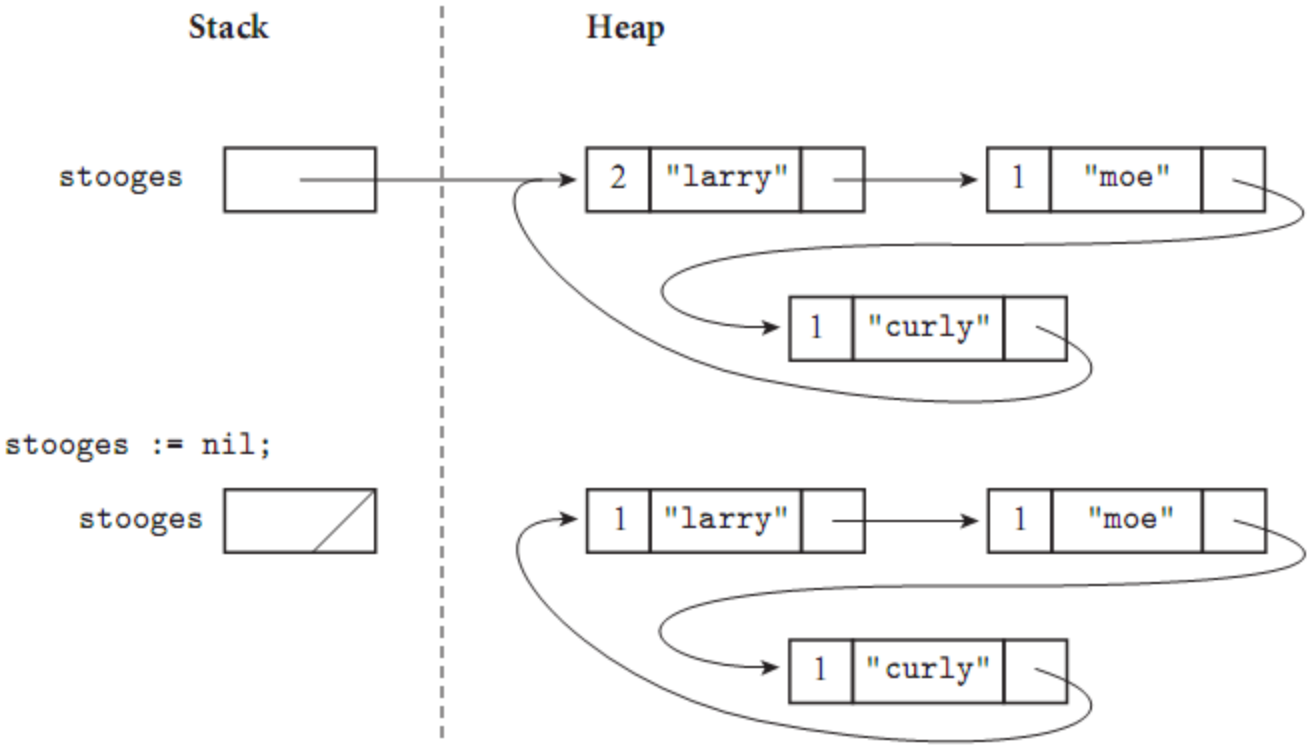
Reference Counter



- Reference counters: maintain a counter in every cell that store the number of pointers currently pointing at the cell
  - *Disadvantages*: space required, execution time required, complications for cells connected circularly
  - *Advantage*: it is intrinsically incremental, so significant delays in the application execution are avoided

Copyright © 2012 Addison-Wesley. All rights reserved.

Garbage Collection With Reference Counts



Mark-Sweep

- The run-time system allocates storage cells as requested and disconnects pointers from cells as necessary; mark-sweep then begins
  - Every heap cell has an extra bit used by collection algorithm
  - All cells initially set to garbage
  - All pointers traced into heap, and reachable cells marked as not garbage
  - All garbage cells returned to list of available cells
  - Disadvantages: in its original form, it was done too infrequently. When done, it caused significant delays in application execution. Contemporary mark-sweep algorithms avoid this by doing it more often—called incremental mark-sweep

Copyright © 2012 Addison-Wesley. All rights reserved.

Mark and Sweep Collecting

- Can build on top of malloc/free package
  - Allocate using `malloc` until you "run out of space"
- When out of space:
  - Use extra ***mark bit*** in the head of each block
  - ***Mark:*** Start at roots and set mark bit on each reachable block
  - ***Sweep:*** Scan all blocks and free blocks that are not marked

